Real-Time Behavioral Fraud Detection Engine

Local Docker Implementation Guide — v2.0 (Critique-Revised)

Based on: `Fraud_Detection_Architecture.md` (OSS Edition)
Supersedes: `Fraud_Detection_Local_Docker_v1.0`
Critique Applied: `architecture_critique.md`
Target: Developer workstation running Docker Desktop / Docker Engine with Compose V2
Status: Documentation — No application code yet

Changelog from v1.0

Area	v1.0	v2.0	Rationale
Message Bus	Apache Kafka (KRaft) + Confluent Schema Registry	**Redpanda** (built-in Schema Registry + Console)	Eliminates heavy JVM overhead; single lightweight C++ container
Stream Processing	Apache Flink (JVM)	**Faust** (`faust-streaming`)	Pure Python asyncio; ML/DS-friendly; no JVM cluster locally
Observability Stack	Jaeger + Loki + Prometheus + Grafana (4 containers)	**SigNoz** (all-in-one OTel backend)	Single container for metrics, traces, and logs
Feature Store	Raw Redis HGETALL + manual TTL management	**Feast** (online: Redis, offline: Iceberg)	Abstracts online/offline sync, point-in-time correctness
Compose Structure	Single monolithic `docker-compose.yml`	**Compose Profiles** (`core`, `obs`, `mgmt`)	Developers only boot what they need
Network Fix	`schema-registry` isolated to `act-net` only	Redpanda attached to both `act-net` and `explain-net`	Fixes Avro deserialization failure in `explain-consumer`
Flink Checkpoints	No persistent volume (state lost on crash)	Host volume `flink-checkpoints` mounted	Enables real chaos recovery testing

Area	v1.0	v2.0	Rationale
Simulator	Happy-path only	Anomaly injection documented (velocity bursts, geo-hops)	Triggers all decision paths including `decision.block`

---

1. Purpose & Scope

This document translates the 6-layer production architecture into a runnable local Docker Compose environment. It is the single source of truth for:

- Every Docker service, its configuration, and which agent owns it
- Every Redpanda topic and its parameters
- Every Avro schema and its field definitions
- The startup sequence and health-check gates
- The latency budget adjusted for single-host Docker overhead
- Failure injection procedures and expected observable outcomes

What This IS

A faithful OSS prototype of the production architecture that runs on a single developer machine with substantially reduced resource requirements compared to v1.0. Engineers can develop, integrate, and validate each system layer before any cloud infrastructure is provisioned.

What This IS NOT

- A production deployment. No high-availability, no multi-node clusters.
- A performance benchmark against production SLAs. Latency targets are relaxed to 2× production budget.
- A security-hardened environment. mTLS, RBAC, and Vault are not configured locally.

---

2. Prerequisites

Hardware

Resource	Minimum	Recommended	v1.0 Was
RAM	**10 GB** (8 GB allocated to Docker)	**16 GB**	16 GB min / 32 GB rec

Resource	Minimum	Recommended	v1.0 Was
CPU	4 cores	8 cores	8 cores min
Disk	30 GB free	60 GB SSD	40 GB min
OS	Windows 11 (WSL2), macOS 13+, Ubuntu 22.04+	—	Same

> **Why the lower spec?** Replacing Kafka + Schema Registry (2 heavy JVM processes) with a single Redpanda container, and replacing Flink (JVM cluster) with Faust (Python asyncio process), eliminates the biggest RAM consumers from v1.0.

Software

Tool	Version	Purpose
Docker Desktop / Engine	4.28+ / 26+	Container runtime
Docker Compose	V2 (bundled)	Service orchestration
Python	3.11+	Faust processors, simulator, MLflow scripts, SHAP
`rpk` (Redpanda CLI)	Latest	Topic inspection and management (replaces `kcat`)
`curl`	Any	API health checks
`jq`	1.6+	JSON output formatting

> **Note:** Java 17 is no longer required locally — Faust runs as a pure Python process.

Reserved Ports

Port	Service	v1.0 Equivalent
9092	Redpanda broker	Kafka broker (9092)
9644	Redpanda admin API	Kafka controller (9093)
8081	Redpanda Schema Registry (built-in)	Confluent Schema Registry (8081)
8080	Redpanda Console (UI)	Flink Job Manager UI (8080)
6379	Redis	Redis (6379)
9000	MinIO S3 API	MinIO S3 API (9000)

Port	Service	v1.0 Equivalent
9001	MinIO Console	MinIO Console (9001)
5432	PostgreSQL	PostgreSQL (5432)
5000	MLflow UI	MLflow UI (5000)
3301	SigNoz UI	Grafana (3000)
4317	OTel Collector (gRPC)	OTel Collector gRPC (4317)
4318	OTel Collector (HTTP)	OTel Collector HTTP (4318)
8082	Airflow Web UI	Airflow Web UI (8082)
6566	Feast UI	*(new)*

3. Repository Layout

```
fraud-detection-local/
|
├── docker-compose.yml                    # Master Compose file with profiles
├── Makefile                              # Convenience targets: up, down, reset,
logs, status
├── .env                                  # Environment variable overrides
|
├── config/
|   ├── redpanda/
|   |   ├── topics.sh                     # Topic creation script (rpk commands)
|   |   └── redpanda.yaml                 # Redpanda broker/tuning overrides
|   ├── redis/
|   |   └── redis.conf                    # AOF persistence, keyspace
notifications
|   ├── minio/
|   |   └── bootstrap.sh                  # Bucket creation script
|   ├── prometheus/                        # Kept for SigNoz scrape config if
needed
|   |   └── alert-rules.yml
|   ├── otel/
|   |   └── otel-collector-config.yaml    # Receivers, processors; exports to
SigNoz
|   ├── feast/
```

```
|   |   ├── feature_store.yaml            # Feast registry, online (Redis),
offline (Iceberg)
|   |   └── features.py                   # Feature view definitions
|   └── signoz/
|       └── otel-collector-config.yaml    # SigNoz internal collector config
|
├── schemas/
|   ├── TransactionEvent.avsc             # Raw input schema
|   ├── FeatureVector.avsc                # Assembled at inference
|   └── DecisionEnvelope.avsc            # Decision output schema
|
├── models/
|   └── README.md                         # How to register an ONNX model in
MLflow
|
├── services/
|   ├── feature-store-writer/             # Faust agent: consumes feature.update →
writes Feast/Redis
|   ├── explain-consumer/                 # Faust agent: consumes cold path →
writes Iceberg
|   ├── fraud-processor/                  # Faust agent: hot path stream
processing + inference
|   └── drift-monitor/                    # Faust agent: consumes decision.* →
computes PSI / KL
|
├── tools/
|   └── tx-simulator/                     # Transaction event generator (normal +
anomaly modes)
|
├── dags/
|   ├── fraud_label_joiner.py
|   ├── fraud_feature_engineering.py
|   ├── fraud_model_training.py
|   ├── fraud_shadow_deployment.py
|   └── fraud_model_promotion.py
|
└── agents/
    ├── 00-agent-overview.md
    ├── 01-infrastructure-agent.md
    ├── 02-ingestion-agent.md
    ├── 03-stream-processing-agent.md       # Now describes Faust, not Flink
    ├── 04-feature-store-agent.md           # Now describes Feast, not raw Redis
    ├── 05-inference-agent.md
    ├── 06-decision-action-agent.md
```

```
    ├── 07-observability-agent.md          # Now describes SigNoz
    ├── 08-explain-compliance-agent.md
    ├── 09-model-governance-agent.md
    └── 10-testing-validation-agent.md
```

---

4. Docker Compose Architecture

Compose Profiles

Services are split into three profiles. Developers only boot what they need:

```bash
# Core streaming pipeline only (fast iteration on ingestion/processing logic)
docker compose --profile core up

# Add observability
docker compose --profile core --profile obs up

# Full stack including model training/governance
docker compose --profile core --profile obs --profile mgmt up
```

Service	Profile
redpanda	core
redis	core
minio	core
feast	core
feature-store-writer	core
explain-consumer	core
fraud-processor	core
drift-monitor	core
otel-collector	obs
signoz	obs
postgres	mgmt
mlflow	mgmt
airflow-webserver	mgmt
airflow-scheduler	mgmt

Network Topology

```
┌──────────────────────────────────────────────────────────────────────┐
│  act-net (hot path — latency critical)                               │
│                                                                      │
│  redpanda (broker + schema-registry built-in)                        │
│    │                                                                 │
│    ▼                                                                 │
│  fraud-processor (Faust asyncio)                                     │
│    │    └──► redis  ◄─── feature-store-writer (Faust)               │
│    │              └──► feast (online store abstraction)              │
│    ▼                                                                 │
│  [inference sidecar — ONNX Runtime in-process via Faust]             │
└──────────────────────────────────────────────────────────────────────┘
                                  │
```

```
                    (feature.update topic)
                                |
┌──────────────────────────────────────────────────────────────────┐
│  explain-net (cold path — async, seconds SLA)                    │
│                                                                  │
│  redpanda ──► explain-consumer (Faust) ──► minio (Iceberg tables)  │
│               │                                                  │
│               └──► drift-monitor (Faust) ──► otel-collector      │
└──────────────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────────────┐
│  obs-net (observability)                                         │
│                                                                  │
│  otel-collector ──► signoz (metrics + traces + logs in one UI)   │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────────────┐
│  mgmt-net (management / training)                                │
│                                                                  │
│  postgres ◄─── mlflow ──► minio (artifact store)                 │
│     └──────────── airflow-scheduler + airflow-webserver          │
└──────────────────────────────────────────────────────────────────┘
```

Network fix from v1.0: `redpanda` is attached to both `act-net` and `explain-net`. In v1.0, `schema-registry` was only on `act-net`, blocking the `explain-consumer` from deserializing Avro payloads. Since Redpanda's Schema Registry is built into the broker, attaching the single `redpanda` container to both networks resolves this entirely.

Service-to-Network Membership

Service	act-net	explain-net	obs-net	mgmt-net
redpanda	✓	✓	—	—
fraud-processor	✓	✓	✓	—
redis	✓	—	✓	—
feast	✓	—	✓	—
minio	—	✓	—	✓
postgres	—	—	—	✓

Service	act-net	explain-net	obs-net	mgmt-net
mlflow	—	—	—	✓
feature-store-writer	✓	—	✓	—
explain-consumer	—	✓	✓	—
drift-monitor	—	✓	✓	—
otel-collector	✓	✓	✓	—
signoz	—	—	✓	—
airflow-webserver	—	—	—	✓
airflow-scheduler	—	—	—	✓

5. Service Reference

5.1 redpanda

Property	Value
Image	`redpandadata/redpanda:v24.1`
Mode	Single-node (replaces Kafka KRaft + Confluent Schema Registry)
Ports	`9092` (Kafka-compatible broker), `9644` (admin), `8081` (Schema Registry), `8080` (Redpanda Console)
Network	act-net, explain-net
Profile	core
Owner	Infrastructure Agent + Ingestion Agent
Health check	`rpk cluster health --brokers localhost:9092`
Startup log	`Successfully started Redpanda!`
Key env vars	`REDPANDA_RPC_SERVER_LISTEN_ADDRESS=0.0.0.0`, `REDPANDA_KAFKA_ADDRESS=0.0.0.0:9092`
Resource limit	`mem_limit: 2g`, `cpus: 2.0`

Property	Value
Volumes	`redpanda-data:/var/lib/redpanda/data`
Local divergence	Single-node. Production: 3+ broker cluster with separate tiered storage.
v1.0 replaces	`kafka` + `schema-registry` (2 containers → 1)

Why Redpanda: Written in C++; boots in under 1 second; includes a built-in Schema Registry and a developer UI (Redpanda Console) in a single lightweight container. Eliminates the two heaviest JVM processes in the v1.0 stack.

5.2 fraud-processor (Faust)

Property	Value
Image	`python:3.11-slim` (custom build with `faust-streaming`, `onnxruntime`, `fastavro`)
Ports	`6066` (Faust web UI)
Network	act-net, explain-net, obs-net
Profile	core
Owner	Stream Processing Agent
Depends on	redpanda (healthy), redis (healthy)
Health check	`curl -sf http://localhost:6066/`
Entrypoint	`faust -A fraud_processor worker -l info`
Volumes	`./services/fraud-processor:/app`, `faust-checkpoints:/var/lib/faust`
Resource limit	`mem_limit: 1g`, `cpus: 2.0`
Local divergence	Single worker process. Production: K8s-deployed Faust workers with horizontal scaling.
v1.0 replaces	`flink-jobmanager` + `flink-taskmanager` (2 JVM containers → 1 Python process)

Why Faust: Stateful stream processing in pure Python using `asyncio`. No JVM cluster, no separate JobManager/TaskManager split, no Java compilation step. ML engineers write streaming logic alongside their PyTorch/Pandas/ONNX code natively.

Faust Checkpoint Volume: A host volume `faust-checkpoints:/var/lib/faust` is mounted so windowed state (transaction counts, spend sums) survives container restarts during chaos testing. This was a gap in v1.0 Flink setup where no checkpoint volume was mounted.

5.3 redis

Property	Value
Image	`redis:7.2-alpine`
Ports	`6379`
Network	act-net, obs-net
Profile	core
Owner	Feature Store Agent (via Feast)
Health check	`redis-cli ping`
Startup log	`Ready to accept connections`
Config	`redis.conf` mounted; AOF enabled; keyspace notifications: `Ex`
Resource limit	`mem_limit: 512m`
Volumes	`redis-data:/data`
Local divergence	Standalone. Production: Redis Cluster (3+ primaries, 3 replicas).

5.4 feast

Property	Value
Image	`python:3.11-slim` (custom build with `feast[redis]`, `pyarrow`)
Ports	`6566` (Feast UI)
Network	act-net, obs-net
Profile	core

Property	Value
Owner	Feature Store Agent
Depends on	redis (healthy), minio (healthy)
Health check	`curl -sf http://localhost:6566/`
Config	`config/feast/feature_store.yaml`
v1.0 replaces	Raw `feat:{entity_type}:{entity_id}` Redis HGETALL pattern

Why Feast: Abstracts the complexity of keeping the online store (Redis) and offline store (Iceberg/MinIO) in sync. Ensures point-in-time correctness for ML training. Eliminates manual TTL management, schema versioning, and the `HGETALL` boilerplate in `feature-store-writer`. The `feature_store.yaml` defines the entity (`account_id`) and feature views (velocity, spend, geo) once; Feast handles materialization.

Feast Configuration (`config/feast/feature_store.yaml`):

```yaml
project: fraud_detection
registry: s3://fraud-features-offline/feast-registry.db
provider: local
online_store:
  type: redis
  connection_string: redis:6379
offline_store:
  type: file   # Iceberg integration via custom datasource in production
artifact_registry_uri: s3://fraud-models/feast/
```

5.5 minio

Property	Value
Image	`minio/minio:RELEASE.2024-04-06T05-26-02Z`
Ports	`9000` (S3 API), `9001` (console)
Network	explain-net, mgmt-net
Profile	core
Owner	Feature Store Agent

Property	Value
Health check	`curl -sf http://localhost:9000/minio/health/live`
Resource limit	`mem_limit: 512m`
Volumes	`minio-data:/data`
Buckets	`fraud-features-offline`, `fraud-audit-logs` (WORM), `fraud-models`, `fraud-lineage`

5.6 feature-store-writer

Property	Value
Image	`python:3.11-slim` (custom build with `faust-streaming`, `feast[redis]`)
Network	act-net, obs-net
Profile	core
Owner	Feature Store Agent
Description	Faust agent consuming `feature.update` topic; calls `feast.push()` to online store (Redis)

5.7 explain-consumer

Property	Value
Image	`python:3.11-slim` (custom build with `faust-streaming`, `fastavro`, `pyiceberg`)
Network	explain-net, obs-net
Profile	core
Owner	Explain / Compliance Agent
Description	Faust agent consuming cold path decisions; writes DecisionEnvelope + SHAP values to Iceberg/MinIO

Property	Value
v1.0 network fix	Now on `explain-net` which has access to `redpanda` (Schema Registry included). No separate schema-registry container needed.

5.8 drift-monitor

Property	Value
Image	`python:3.11-slim` (custom build with `faust-streaming`, `scipy`, `opentelemetry-sdk`)
Network	explain-net, obs-net
Profile	core
Owner	Model Governance Agent
Description	Faust agent consuming `decision.*` topics; computes PSI / KL divergence; emits metrics to OTel Collector

5.9 otel-collector

Property	Value
Image	`otel/opentelemetry-collector-contrib:0.99.0`
Ports	`4317` (gRPC), `4318` (HTTP)
Network	act-net, explain-net, obs-net
Profile	obs
Owner	Observability Agent
Config	`config/otel/otel-collector-config.yaml`
Exporter	Forwards all telemetry to SigNoz via OTLP (`signoz:4317`)

5.10 signoz

Property	Value
Image	`signoz/signoz:latest` (all-in-one)
Ports	`3301` (UI — metrics, traces, logs)
Network	obs-net
Profile	obs
Owner	Observability Agent
Resource limit	`mem_limit: 2g`
Volumes	`signoz-data:/var/lib/signoz`
Default credentials	`admin@fraud.local / fraud-local`
v1.0 replaces	`jaeger` + `loki` + `prometheus` + `grafana` (4 containers → 1)

Why SigNoz: Provides metrics, distributed tracing, and logs in a single backend and UI natively built on OpenTelemetry. Eliminates the need for separate Jaeger (traces), Loki (logs), Prometheus (metrics), and Grafana (dashboards) containers — the four-container observability stack in v1.0. SigNoz accepts OTLP natively, so the OTel Collector config requires only a single OTLP exporter.

Trace queries: Navigate to SigNoz UI at `http://localhost:3301`, select service `fraud-inference`, filter by span attribute `latency_total_us`. Equivalent to v1.0 Jaeger UI at port `16686`.

5.11 postgres

Property	Value
Image	`postgres:16-alpine`
Ports	`5432`
Network	mgmt-net
Profile	mgmt
Owner	Infrastructure Agent

Property	Value
Health check	`pg_isready -U fraud`
Databases	`mlflow`, `airflow`
Resource limit	`mem_limit: 512m`
Volumes	`postgres-data:/var/lib/postgresql/data`

5.12 mlflow

Property	Value
Image	`ghcr.io/mlflow/mlflow:v2.12.1`
Ports	`5000`
Network	mgmt-net
Profile	mgmt
Owner	Inference Agent
Depends on	postgres (healthy), minio (healthy)
Health check	`curl -sf http://localhost:5000/health`
Key env vars	`MLFLOW_BACKEND_STORE_URI=postgresql://fraud:fraud@postgres/mlflow`, `MLFLOW_ARTIFACT_ROOT=s3://fraud-models/`

5.13 airflow-webserver & airflow-scheduler

Property	Value
Image	`apache/airflow:2.9.1-python3.11`
Ports	`8082` (webserver only)
Network	mgmt-net
Profile	mgmt

Property	Value
Owner	Model Governance Agent
Depends on	postgres (healthy), minio (healthy)
Resource limit	`mem_limit: 1g` each
Volumes	`airflow-dags:/opt/airflow/dags`
Default credentials	`admin / fraud-local`

6. Startup Sequence

Boot groups map to Compose profiles. Services within a profile start in dependency order.

```
── PROFILE: core ──────────────────────────────────────────────────────

GROUP 1 — Foundation (no dependencies)
  minio
  redis

GROUP 2 — Message Bus (no Zookeeper, no separate Schema Registry)
  redpanda   ← Broker + Schema Registry + Console in one container

GROUP 3 — Feature Store Abstraction
  feast      ← Depends on redis + minio

GROUP 4 — Stream Processors (Faust workers)
  fraud-processor       ← Depends on redpanda + redis + feast
  feature-store-writer  ← Depends on redpanda + feast
  explain-consumer      ← Depends on redpanda + minio
  drift-monitor         ← Depends on redpanda

── PROFILE: obs ───────────────────────────────────────────────────────

GROUP 5 — Observability
  otel-collector   ← Forwards to signoz
  signoz           ← All-in-one: metrics + traces + logs

── PROFILE: mgmt ──────────────────────────────────────────────────────

GROUP 6 — Management & Training (heavy — boot separately)
  postgres
  mlflow             ← Depends on postgres + minio
  airflow-scheduler  ← Depends on postgres + minio
  airflow-webserver  ← Depends on postgres + minio
```

Makefile targets:

```
make core          # Start core profile only
make obs           # Start core + obs profiles
make up            # Start all profiles (core + obs + mgmt)
make down          # Stop all services (preserve volumes)
make reset         # Stop + remove volumes (clean slate)
make logs          # Tail logs for all running services
make status        # docker compose ps with health status
make topics        # Create all Redpanda topics (run after redpanda healthy)
make schemas       # Register all Avro schemas via Redpanda Schema Registry
make buckets       # Create MinIO buckets (run after minio healthy)
make feast-apply   # Apply Feast feature store definitions
make sim-normal    # Run tx-simulator in normal mode (happy path)
make sim-anomaly   # Run tx-simulator in anomaly mode (triggers BLOCK/REVIEW)
```

---

7. Redpanda Topic Inventory

All topics use `account_id` as the partition key unless noted. Topics are created via `rpk topic create` in `config/redpanda/topics.sh`.

Topic	Partitions	Retention	Replication	Consumer Groups	Description
`tx.raw.hot`	16	2 hours	1 (local)	`faust-act-path`	Hot path — real-time inference
`tx.raw.cold`	8	7 days	1 (local)	`faust-explain-path`	Cold path — offline lake, audit
`feature.update`	8	1 hour	1 (local)	`feature-store-writer`	Feature delta events → Feast/Redis
`decision.block`	4	30 days	1 (local)	`fraud-case-mgmt`, `drift-monitor`, `explain-consumer`	Blocked transactions
`decision.allow`	4	24 hours	1 (local)	`drift-monitor`	Approved transactions
`decision.review`	4	30 days	1 (local)	`human-review`, `drift-monitor`, `explain-consumer`	Flagged for manual review
`decision.degraded`	2	30 days	1 (local)	`fraud-case-mgmt`, `explain-consumer`	Degraded-mode decisions
`tx.dead-letter`	2	7 days	1 (local)	`manual-remediation`	Schema-rejected / shed events

v2.0 partition reduction: Partition counts are reduced from v1.0 (e.g., `tx.raw.hot`: 64 → 16). Locally, high partition counts with a single Redpanda broker and a single Faust worker process provide no throughput benefit and add memory overhead.

Local divergence: Replication factor 1 on all topics. Production: replication factor 3, `min.insync.replicas=2`.

Topic management (rpk):

```bash
# List topics
rpk topic list --brokers localhost:9092

# Consume from a topic (equivalent to v1.0 kcat)
rpk topic consume tx.raw.hot --brokers localhost:9092 --num 10

# Describe topic
rpk topic describe decision.block --brokers localhost:9092
```

---

8. Avro Schema Definitions

Schemas are registered in Redpanda's built-in Schema Registry (port `8081`) using the same Confluent-compatible API as v1.0. No change to schema definitions — only the registry endpoint changes.

Schema registration:

```bash
# Register (identical curl command as v1.0 — Redpanda is Confluent-compatible)
curl -X POST http://localhost:8081/subjects/tx-raw-value/versions \
  -H "Content-Type: application/vnd.schemaregistry.v1+json" \
  -d @schemas/TransactionEvent.avsc

# Verify
curl http://localhost:8081/subjects
```

8.1 TransactionEvent (subject: `tx-raw-value`)

Field	Type	Nullable	Description
`transaction_id`	string	No	UUID v4 — unique per event
`source_system`	string	No	`mobile`, `web`, `gateway-{name}`
`account_id`	string	No	Primary partition key
`device_id`	string	Yes	Null for web/gateway sources

Field	Type	Nullable	Description
`ip_address`	string	Yes	IPv4 or IPv6
`merchant_id`	string	No	
`merchant_category`	string	No	MCC code
`amount_cents`	long	No	Transaction amount in smallest currency unit
`currency`	string	No	ISO 4217 code
`country_code`	string	No	ISO 3166-1 alpha-2
`event_timestamp`	long	No	Unix epoch milliseconds (event-time)
`ingestion_timestamp`	long	No	Unix epoch milliseconds (processing-time)
`source_system_hash`	string	No	SHA-256 of `transaction_id + source_system` (dedup key)

8.2 FeatureVector (subject: `feature-vector-value`)

Field	Type	Nullable	Description
`entity_id`	string	No	`account:{account_id}`
`schema_version`	string	No	Must match model's pinned schema version
`feature_timestamp`	long	No	Timestamp of most recent contributing event
`txn_count_1m`	double	No	Transaction count in last 1 minute
`txn_count_5m`	double	No	Transaction count in last 5 minutes
`txn_count_15m`	double	No	Transaction count in last 15 minutes
`txn_count_1h`	double	No	Transaction count in last 1 hour
`txn_count_24h`	double	No	Transaction count in last 24 hours
`spend_sum_1m`	double	No	Sum of amounts in last 1 minute
`spend_sum_5m`	double	No	Sum of amounts in last 5 minutes
`spend_sum_1h`	double	No	Sum of amounts in last 1 hour

Field	Type	Nullable	Description
`spend_avg_24h`	double	No	Rolling spend average over 24 hours
`distinct_merchants_1h`	int	No	Unique merchants in last 1 hour
`distinct_countries_24h`	int	No	Unique countries in last 24 hours
`distinct_devices_24h`	int	No	Unique device IDs in last 24 hours
`session_txn_count`	int	No	Transactions in current session (30m inactivity gap)
`session_duration_s`	double	No	Current session duration in seconds
`low_confidence`	boolean	No	True if cold-start or TTL expiry (prior profile used)

8.3 DecisionEnvelope (subject: `decision-value`)

Field	Type	Nullable	Description
`transaction_id`	string	No	Correlates to TransactionEvent
`source_correlation_id`	string	No	End-to-end trace ID
`decision`	enum	No	`ALLOW`, `REVIEW`, `BLOCK`, `DEGRADED_BLOCK`
`risk_score`	double	No	0.0 (low risk) – 1.0 (high risk)
`confidence_lower`	double	No	Lower bound of confidence band
`confidence_upper`	double	No	Upper bound of confidence band
`model_version`	string	No	MLflow model version tag
`feature_schema_version`	string	No	Schema Registry schema version
`processing_ts`	long	No	Unix epoch milliseconds
`latency_ingestion_us`	long	No	Ingestion stage duration (microseconds)
`latency_feature_us`	long	No	Feature hydration duration (microseconds)
`latency_inference_us`	long	No	Inference scoring duration (microseconds)

Field	Type	Nullable	Description
`latency_emission_us`	long	No	Action emission duration (microseconds)
`latency_total_us`	long	No	Sum of all stages (microseconds)
`low_confidence`	boolean	No	True if prior profile used
`flags`	array[string]	No	e.g., `cold_start`, `degraded_mode`, `challenger_scored`

9. Latency Budget — Local Docker

Local target: <60ms P99 (2× production budget — single host, Docker bridge network overhead, no kernel bypass, no NUMA pinning)

Stage	Component	Local P99 Target	Production P99 Target	Notes
Ingestion & Schema Validation	Redpanda + built-in Schema Registry	3–5ms	2–3ms	Faster than Kafka locally due to C++ broker
Internal Bus Transit	Topic → Consumer	1–2ms	1ms	
Feature Retrieval	Feast online store (Redis)	3–5ms	4–6ms	Faster locally (no network hop)
Feature Vector Assembly	In-process Faust	<1ms	<0.5ms	
Inference (ONNX, CPU)	ONNX Runtime in-process	8–15ms	6–10ms	No GPU locally
Decision Envelope Construction	In-process	<1ms	<0.5ms	
Action Emission	Redpanda produce + sync response	3–5ms	2–3ms	
Total		**~19–33ms**	**~16–24ms**	**Local headroom: 27–41ms**

Measuring latency locally:

```bash
# SigNoz traces (replaces Jaeger in v1.0)
# Navigate to: http://localhost:3301 → Traces → Service: fraud-inference
# Filter by: attribute "latency_total_us"

# Metrics query in SigNoz (replaces Prometheus UI)
# Navigate to: http://localhost:3301 → Metrics
# Query: histogram_quantile(0.99, fraud_e2e_latency_ms_bucket)
```

The `latency_trace` fields in `DecisionEnvelope` give per-transaction microsecond breakdowns regardless of UI.

---

10. Feast Feature Store — Configuration & Key Schema

Feast replaces the raw Redis `feat:{entity_type}:{entity_id}` HGETALL pattern from v1.0.

Feature Store YAML (`config/feast/feature_store.yaml`)

```yaml
project: fraud_detection
registry: s3://fraud-features-offline/feast-registry.db
provider: local
online_store:
  type: redis
  connection_string: redis:6379
offline_store:
  type: file
```

Feature View Definition (`config/feast/features.py`)

```python
```

```python
from feast import Entity, FeatureView, Feature, ValueType, RedisOnlineStore
from datetime import timedelta

account = Entity(name="account_id", value_type=ValueType.STRING)

account_velocity = FeatureView(
    name="account_velocity",
    entities=["account_id"],
    ttl=timedelta(hours=72),
    features=[
        Feature(name="txn_count_1m",           dtype=ValueType.DOUBLE),
        Feature(name="txn_count_5m",            dtype=ValueType.DOUBLE),
        Feature(name="txn_count_15m",           dtype=ValueType.DOUBLE),
        Feature(name="txn_count_1h",            dtype=ValueType.DOUBLE),
        Feature(name="txn_count_24h",           dtype=ValueType.DOUBLE),
        Feature(name="spend_sum_1m",            dtype=ValueType.DOUBLE),
        Feature(name="spend_sum_5m",            dtype=ValueType.DOUBLE),
        Feature(name="spend_sum_1h",            dtype=ValueType.DOUBLE),
        Feature(name="spend_avg_24h",           dtype=ValueType.DOUBLE),
        Feature(name="distinct_merchants_1h",   dtype=ValueType.INT32),
        Feature(name="distinct_countries_24h",  dtype=ValueType.INT32),
        Feature(name="distinct_devices_24h",    dtype=ValueType.INT32),
        Feature(name="session_txn_count",       dtype=ValueType.INT32),
        Feature(name="session_duration_s",      dtype=ValueType.DOUBLE),
        Feature(name="schema_version",          dtype=ValueType.STRING),
        Feature(name="feature_timestamp",       dtype=ValueType.INT64),
    ],
)
```

Feature Retrieval in fraud-processor

```python
# In Faust agent — replaces raw HGETALL
feature_vector = store.get_online_features(
    features=["account_velocity:txn_count_1m", "account_velocity:spend_sum_1h", ...]
    entity_rows=[{"account_id": event.account_id}]
).to_dict()

# On MISS: Feast returns None values → set low_confidence=True
low_confidence = any(v is None for v in feature_vector.values())
```

On MISS (new entity or expired TTL): Feast returns `None` for missing features. The processor sets `low_confidence=True` in the FeatureVector and triggers an async backfill from the offline store — same semantics as v1.0, but managed by Feast rather than custom Redis logic.

---

11. MinIO Bucket & Iceberg Table Inventory

Unchanged from v1.0. MinIO configuration and bucket lifecycle policies are identical.

Buckets

Bucket	Purpose	WORM	Lifecycle
`fraud-features-offline`	Historical feature snapshots (Iceberg) + Feast registry	No	90 days
`fraud-audit-logs`	Immutable decision + explain records (Iceberg)	**Yes**	7 years
`fraud-models`	MLflow model artifacts (ONNX files, metadata) + Feast artifacts	No	Indefinite
`fraud-lineage`	OpenLineage facets, data provenance	No	2 years

Iceberg Tables

Table	Partition Spec	Description
`fraud_features.entity_features`	`date(feature_timestamp), entity_id_prefix(4)`	Historical feature snapshots from stream
`fraud_audit.decision_log`	`date(processing_ts), decision`	Every DecisionEnvelope produced
`fraud_audit.explain_log`	`date(processing_ts)`	SHAP values + counterfactuals (async)
`fraud_training.labeled_dataset`	`date(label_join_ts)`	Point-in-time correct training rows

---

12. Transaction Simulator — Normal & Anomaly Modes

The v1.0 simulator only generated happy-path data, which never triggered `decision.block` or `decision.review` topics. v2.0 documents both modes explicitly.

Normal Mode (baseline)

```bash
make sim-normal
# OR directly:
python tools/tx-simulator/simulate.py --mode normal --tps 100 --duration 300
```

Generates realistic transactions: varied merchants, amounts within normal range, single country, consistent device.

Anomaly Mode (triggers BLOCK / REVIEW)

```bash
make sim-anomaly
# OR directly:
python tools/tx-simulator/simulate.py --mode anomaly --scenario all
```

Scenario flag	What it generates	Expected decision
`--scenario velocity_burst`	50+ transactions in 60 seconds for one account	`decision.block`
`--scenario geo_hop`	Transactions from 3+ countries within 10 minutes	`decision.block`
`--scenario amount_spike`	Single transaction 20× account's `spend_avg_24h`	`decision.review`
`--scenario new_device`	Transaction from new `device_id` + high amount	`decision.review`
`--scenario cold_start`	New `account_id` never seen before	`low_confidence=true` in envelope
`--scenario schema_corrupt`	Missing required field `amount_cents`	`tx.dead-letter`
`--scenario all`	Interleaves all of the above with normal traffic	All decision paths

Validate the full graph:

```bash
# Monitor all decision topics simultaneously
rpk topic consume decision.block decision.allow decision.review decision.degraded \
  --brokers localhost:9092 --follow
```

---

13. Failure Injection Procedures

These procedures test the same failure modes as v1.0, adapted for the v2.0 service names.

13.1 Feature Store Unavailable

```bash
```

```
docker pause fraud-detection-local-redis-1
# Expected: Feast returns None features; fraud-processor uses prior_profile defaults
# Expected: DecisionEnvelope has low_confidence=true, flags includes "degraded_mode"
# Expected: SigNoz alert "FeatureStoreReadHighLatency" fires within 30s
docker unpause fraud-detection-local-redis-1
# Expected: Backlog clears; low_confidence returns false for known entities
```

13.2 Stream Processor Crash (replaces Flink TaskManager kill)

```bash
docker kill fraud-detection-local-fraud-processor-1
# Expected: Consumer lag accumulates on tx.raw.hot
# Expected: SigNoz alert "FaustWorkerDown" fires
docker compose --profile core up -d fraud-processor
# Expected: Faust reloads from faust-checkpoints volume; windowed state recovered
# Expected: Lag clears; no events permanently lost (Redpanda retention provides repl
```

> **Why this works:** The `faust-checkpoints` host volume mount preserves RocksDB state across container restarts. This was broken in v1.0 because no Flink checkpoint volume was configured.

13.3 Schema Mismatch

```bash
# Produce a malformed event directly via rpk (replaces v1.0 kcat command)
echo '{"transaction_id":"test","amount_cents":100}' | \
  rpk topic produce tx.raw.hot --brokers localhost:9092
# Expected: Redpanda Schema Registry rejects; event routes to tx.dead-letter
# Expected: No inference attempted; no BLOCK/ALLOW emitted
```

13.4 Downstream Consumer Down

```bash
docker pause fraud-detection-local-explain-consumer-1
# Expected: decision.block topic depth increases (consumer lag visible in Redpanda C
# Expected: Act path UNAFFECTED — decisions still emit to topic
docker unpause fraud-detection-local-explain-consumer-1
# Expected: Lag clears; explain records written to Iceberg
```

13.5 Message Bus Restart (new in v2.0)

```bash
docker restart fraud-detection-local-redpanda-1
# Expected: Redpanda boots in <2 seconds (C++ startup vs JVM 30-60s in v1.0)
# Expected: Schema Registry re-available immediately (built-in — no separate contain
# Expected: Faust workers reconnect automatically via configured retry policy
```

---

14. Compliance Checklist — Local Validation

EU AI Act / DORA Requirement	Local Implementation	Validated By
Auditability of AI decisions	`fraud_audit.decision_log` Iceberg table	Explain Agent
Explainability (SHAP)	`fraud_audit.explain_log` with SHAP values	Explain Agent
Human oversight (`REVIEW` class)	Decision routes to `decision.review` topic	Decision Agent
Data lineage	OpenLineage facets in `fraud-lineage` bucket	Governance Agent
Model documentation (model card)	MLflow experiment description + tags	Inference Agent
Bias / fairness report	Drift Monitor PSI by merchant category proxy (Faust + SigNoz)	Governance Agent
Incident logging	SigNoz alert → annotation (replaces Prometheus → Grafana)	Observability Agent
Right to explanation (GDPR Art. 22)	`GET /audit/transaction/{id}` query API	Explain Agent
Data retention (7 years)	`fraud-audit-logs` bucket lifecycle policy	Feature Store Agent
Rollback capability	MLflow re-tag + Faust hot-reload (replaces Flink hot-reload)	Inference Agent

EU AI Act / DORA Requirement	Local Implementation	Validated By
Point-in-time feature correctness	Feast offline store backfill (new — not in v1.0)	Feature Store Agent

15. Resource Allocation Summary

Explicit `deploy.resources.limits` on all services (addresses v1.0 critique gap):

```yaml
# In docker-compose.yml — applied to each service
deploy:
  resources:
    limits:
      memory: <see below>
      cpus: <see below>
```

> **v1.0 had no resource limits**. Without `deploy.resources.limits`, containers compete freely for host memory, causing OOM kills when the JVM heap of Kafka and Flink expands under load.

Document Owner: Lead Application Architect
Implementation: Multi-agent Docker local prototype
Supersedes: Fraud_Detection_Local_Docker v1.0
Critique Applied: architecture_critique.md

Review cycle: After each agent completes their implementation phase
Classification: Internal — Implementation